

StegoVault: Algorithm & Model Documentation

A Technical Reference for Multi-Modal Steganography, Cryptography, and Deep Learning Architectures

Overview: This document provides detailed documentation of the cryptographic, steganographic, and deep learning algorithms implemented within the *StegoVault* platform. The platform supports secure multi-modal information hiding across Image, Text, Audio, and Video files, combining neural representation learning with authenticated symmetric key encryption.

1. Cryptography / Secret Protection Layer

To ensure high security, secret payloads are never embedded as raw text or files. The system enforces state-of-the-art symmetric encryption before any steganographic encoding occurs:

- **AES-GCM (Advanced Encryption Standard in Galois/Counter Mode):** Used for symmetric encryption and decryption of the secret payload. AES-GCM guarantees both confidentiality (encryption) and integrity (authentication) via Galois message authentication codes. The system uses a 256-bit key size.
- **SHA-256 (Secure Hash Algorithm 256-bit):** Serves multiple vital roles in key derivation and validation:
 - Key Derivation:* Derives a 256-bit AES key from the user-provided access key via `SHA-256(access_key)`.
 - File Signature:* Computes a unique signature of the original cover file via `SHA-256(cover_bytes)`.
 - Key Binding:* Binds the access key with the cover file via `SHA-256(access_key + ':' + file_signature)`. This verification hash prevents attackers from decrypting tampered files or using invalid access keys.
- **Cryptographically Secure Randomness:** Access keys are derived from 32 bytes of secure random noise generated by the OS-level entropy pool (via Python's `secrets` module) and encoded in a URL-safe Base64 format.

2. Steganography Payload Container (Packaging)

StegoVault packages the encrypted payload using a custom **EOF-Based Binary Container Strategy**. This allows seamless metadata and payload embedding without corrupting or modifying sensitive media structures which could otherwise render image, audio, or video files unreadable or visually degraded.

Container Byte Layout Structure:

The final stego file is laid out as follow in the file system:

[Original Cover Media Bytes] + [MAGIC Bytes (b'STEGOV1\0')] + [4-byte Metadata Length] + [JSON Metadata] + [AES-GCM Ciphertext]

During **decryption (decoding)**, the system reads the file from the end, locating the marker `STEGO1`. It splits the container, extracts the metadata to retrieve the key binding and nonce, verifies the binding against the provided access key, and then decrypts the ciphertext payload.

3. Deep Learning Modality Models

For deep learning-based information hiding, StegoVault includes modular PyTorch neural network architectures tailored for each modality. The networks introduce subtle, high-dimensional perturbations to embed payload signatures that are difficult to detect via traditional statistical steganalysis.

Modality	Deep Learning Architecture	Key Layers & Activations	Perturbation / Math
Image	2D Convolutional Neural Network (2D CNN)	3 Layers: Conv2d(3,32,k3) -> ReLU -> Conv2d(32,32,k3) -> ReLU -> Conv2d(32,3,k1)	Cover + 0.01 * CNN(Cover) Clamped [0.0, 1.0]
Audio	1D Convolutional Neural Network (1D CNN)	3 Layers: Conv1d(1,16,k7) -> GELU -> Conv1d(16,16,k5) -> GELU -> Conv1d(16,1,k1)	Waveform + 0.005 * CNN(Waveform) Clamped [-1.0, 1.0]
Video	3D Convolutional Neural Network (3D CNN)	3 Layers: Conv3d(3,8,k3) -> ReLU -> Conv3d(8,8,k3) -> ReLU -> Conv3d(8,3,k1)	Frames + 0.01 * CNN(Frames) Clamped [0.0, 1.0]
Text	Transformer Encoder Sequence Model	2-Layer TransformerEncoderLayer (d_model=64, nhead=4, d_ff=128)	Transformer-encoded representations

Model Execution Notes:

4. Post-Processing & Quality Enhancement Algorithms

Image Quality Enhancement:

- **LANCZOS Resampling:** Low resolution cover images (under 2048px) are upscaled by a factor of 2 using Lanczos interpolation.
- **Detail Filtering:** Applies a high-pass detail filter to recover edge textures and sharp boundaries.
- **Visual Statistics Adjustments:** Custom tuning adjustments are applied for Sharpness (1.5x), Contrast (1.15x), and Color/Saturation (1.1x) to keep the images visually vibrant and offset any encoder distortions.

Video Quality Enhancement:

- **Bicubic Resizing:** Resizes video frames up to 1.5x or 2.0x using cubic interpolation.
- **Bilateral Filtering:** Applies edge-preserving smoothing (via OpenCV's `bilateralFilter`) to suppress high-frequency encoder noise while maintaining crisp outlines.
- **2D Convolutional Sharpening:** Filters each frame with a custom 3x3 sharpening kernel.
- **Weighted Alpha-Blending:** Blends the bilateral denoised frames and sharpened frames using a 70% to 30% ratio (`cv2.addWeighted`) to maintain smooth but clean edges.